

CUDA Programming Notes

GPU Architecture, Parallelism & Optimization

1. CUDA Basics

__global__ Function (Kernel)

A function marked `__global__` runs on the GPU. It is called a **kernel** and is launched from the CPU. Many instances execute in parallel.

- Runs on the **GPU** (device)
- Launched from the **CPU** (host)
- Executes in **massive parallelism** — thousands of threads simultaneously
- `threadIdx.x` — unique ID of each thread within its block

Launching a Kernel

Kernel launch syntax:

```
myKernel <<< numBlocks, threadsPerBlock >>> (args);
```

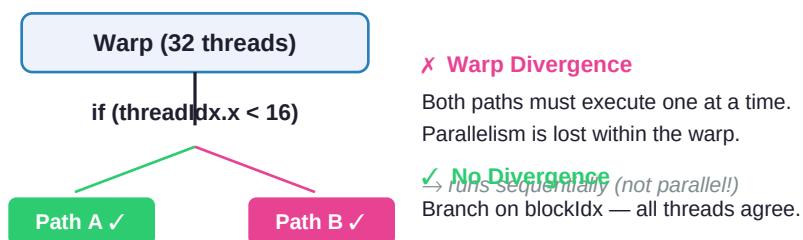
of blocks # of threads per block

cudaDeviceSynchronize()

- When a kernel is launched, the GPU runs **asynchronously** — out of sync with the CPU.
- Call `cudaDeviceSynchronize()` to make the CPU wait for the GPU to finish.
- **Don't overuse it** — kernels launched in sequence will start in order regardless.

2. Parallelising Work — Warp Divergence

A **warp** is a group of 32 threads that execute the same instruction simultaneously (SIMT). When threads in the same warp take **different branches**, the GPU must run each path sequentially — this is **warp divergence** and kills parallelism.



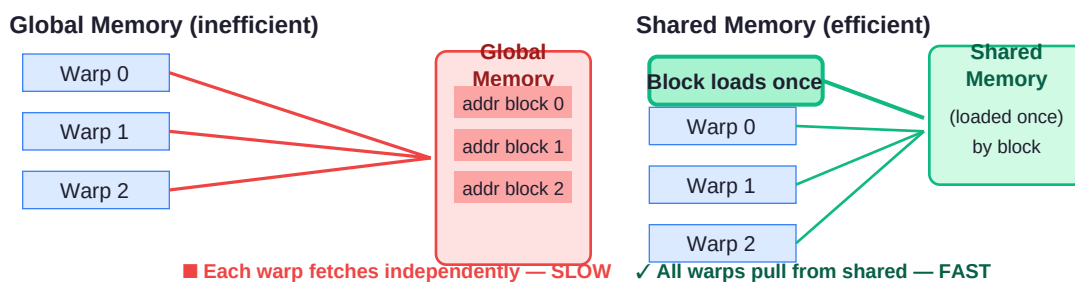
Best Practices

- Avoid heavy branching inside kernels.
- Try to keep all threads in a warp on the **same path**.
- Edge-case bounds checks are fine — minimal divergence.
- **Avoid**: branching on **threadIdx** values that split the warp (e.g., `threadIdx.x < 16`).
- **OK**: branching on **blockIdx** — the entire block (all warps) takes the same path.

3. Memory — Coalescing, Global & Shared

Memory Coalescing

Threads in the same warp should access **consecutive memory addresses**. This lets the GPU combine (coalesce) them into a single memory transaction — similar to cache-line locality on a CPU.



Memory Hierarchy

Ordered from fastest/smallest to slowest/largest:

Memory Type	Latency ← fast ————— slow →	Scope
Registers ~1 cycle		1 thread
Shared Memory ~5 cycles		All threads in block
L1 / L2 Cache ~30 cycles		Automatic / local
Global Memory ~500 cycles		Entire GPU

↑ fastest, smallest
↓ slowest, largest

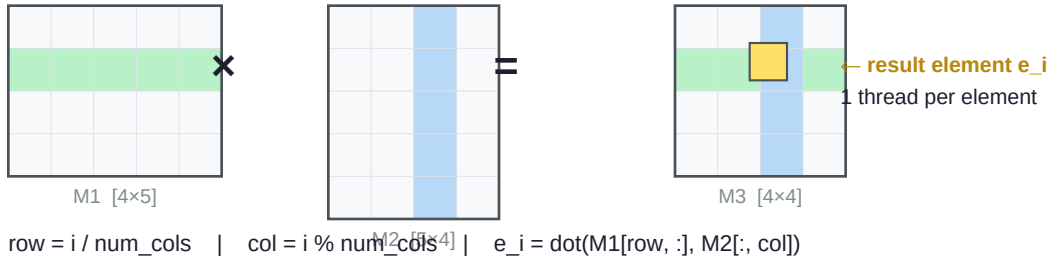
- **Registers** — Single values used by one thread. Fastest.
- **Shared Memory** — Data reused within a block. Try to optimise for this.
- **L1 / L2 Cache** — Automatic, local data. No explicit management needed.
- **Global Memory** — Data used only once, or persistent across blocks. Slowest.

4. Matrix Multiplication Design

Basic Design

Inputs and output:

- M1: $[M \times K]$ — flattened row-major vector
- M2: $[K \times N]$ — flattened row-major vector
- M3: $[M \times N]$ — output vector
- Total threads: $M \times N$ — one per output element



Kernel Design — Simple Version

Each thread computes one output element:

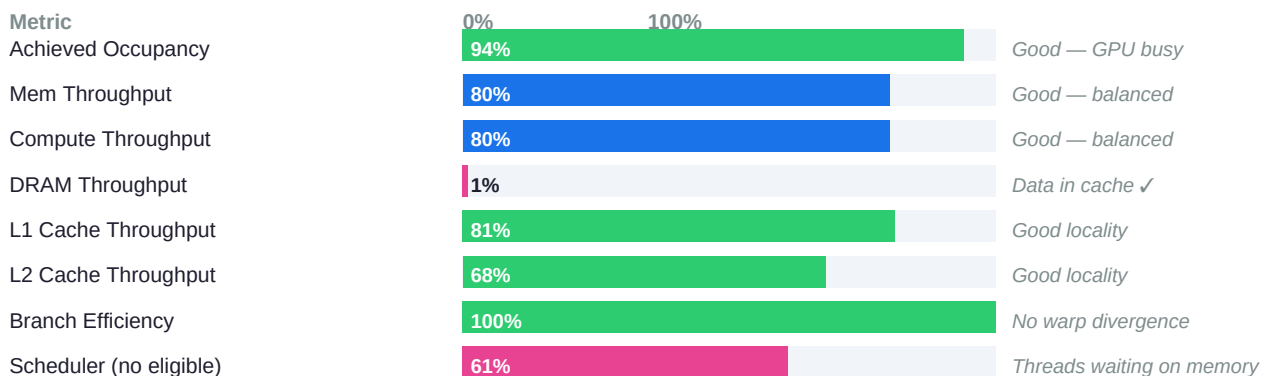
- Inputs: pointers to M1, M2, M3 + dimension values
- Thread index i maps to: $\text{row} = i / \text{num_cols}$, $\text{col} = i \% \text{num_cols}$
- Compute: $e_i = \text{dot}(M1[\text{row}, :], M2[:, \text{col}])$
- Max 1024 threads per block $\rightarrow \text{num_blocks} = (\text{elements} + \text{threads} - 1) / \text{threads}$ (ceiling division)

Host-side Flow

1. Check dimensions — $\text{cols}(M1) == \text{rows}(M2)$
2. Allocate device memory (`cudaMalloc`), set vector pointer to `.data()`
3. Copy M1, M2 to device (`cudaMemcpy`)
4. Launch kernel
5. Copy M3 back to host, free device memory

5. Nsight Profiler Observations — Baseline (512x512)

Profiling the naive matrix multiply with no optimisations:



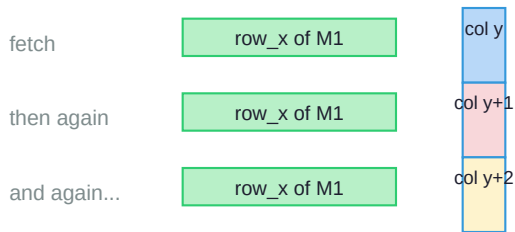
Key Issue: Scheduler — 61% No Eligible Warps

Threads issue instructions only **38% of cycles**. The rest of the time they are stalling, waiting for memory fetches to return from global memory.

Root Cause: Redundant Memory Fetches

Each output column requires the **same row of M1** — but the naive kernel re-fetches it from global memory every time. Uncoalesced memory access pattern (`dev_m1[row*K + i]`) means addresses are spread far apart.

Problem: Same row of M1 fetched repeatedly for each column of M2



Solution

1. Load tile of M1 into Shared Memory
2. Load tile of M2 into Shared Memory
3. All warps in block reuse the tile
→ Memory Tiling / Tiled MatMul

6. Next Steps

• Shared Memory

Load a tile of M1 and M2 into shared memory once per block. All warps in the block reuse the tile without going back to slow global memory.

• Memory Tiling (Tiled MatMul)

Divide the matrices into tiles of size $T \times T$. Each block handles one output tile. Loop over the K dimension in T-sized steps, staging data through shared memory. Reduces global memory traffic from $O(M \cdot N \cdot K)$ fetches to $O(M \cdot N \cdot K / T)$.

• Ceiling Division Across Blocks

Learn robust patterns for `num_blocks = (N + T - 1) / T` — ensure all elements are covered even when N is not divisible by T.

End of notes.